SERVI-X Documentacion Final

Jose Alvarez Marin

Facultad de Ingenieria, Corporacion Universitaria de Sabaneta

Integracion de Aplicaciones y Sistemas

Dr. Mauricio Fabian Agudelo

Noviembre de 2026

Resumen

Este documento presenta el modelado tecnico detallado para el modulo SERVI-X, responsable de la gestion integral de Peticiones, Quejas, Reclamos y Sugerencias (PQRS) dentro del ecosistema de microservicios INTEGRAX. A traves de un enfoque de diseno riguroso que abarca el Modelo de Datos (Entidad-Relacion) y el modelado UML (Casos de Uso, Clases y Secuencia), se define la estructura logica, el almacenamiento y el comportamiento dinamico del sistema. El diseno propuesto garantiza el aislamiento de datos propio de la arquitectura del proyecto, prioriza la trazabilidad inmutable de las actuaciones (auditoria) y establece los mecanismos de integracion asincrona y sincrona via API REST con los modulos externos CRM-X, SALES-X y LOGISTI-X.

Alcance vigente del modulo: en esta fase, SERVI-X opera de forma aislada, con frontend, API REST y base de datos PostgreSQL ya funcionando de extremo a extremo (ver seccion 10). Las integraciones con los servicios externos del ecosistema (CRM-X, SALES-X y LOGISTI-X) estan disenadas en el modelo de datos mediante llaves foraneas logicas (*cliente_id*, *pedido_id*, *despacho_id*), pero su consumo real via HTTP/REST queda **planificado para una fase posterior**.

Diagrama de Gantt y Metodologia de Trabajo

El proyecto se gestiona con una metodologia hibrida: el cronograma general de INTEGRAX fija hitos semanales de entrega obligatoria (viernes de cada semana, del 21 de agosto al 18 de septiembre de 2026 segun el cronograma oficial del curso), mientras que el trabajo dentro de cada semana se organiza de forma agil mediante ramas de feature en Git (*feature/** -> *dev* -> *main*), con aprobacion explicita antes de cada integracion. Esto permite entregar incrementos funcionales verificables en cada hito sin perder la trazabilidad de los cambios.

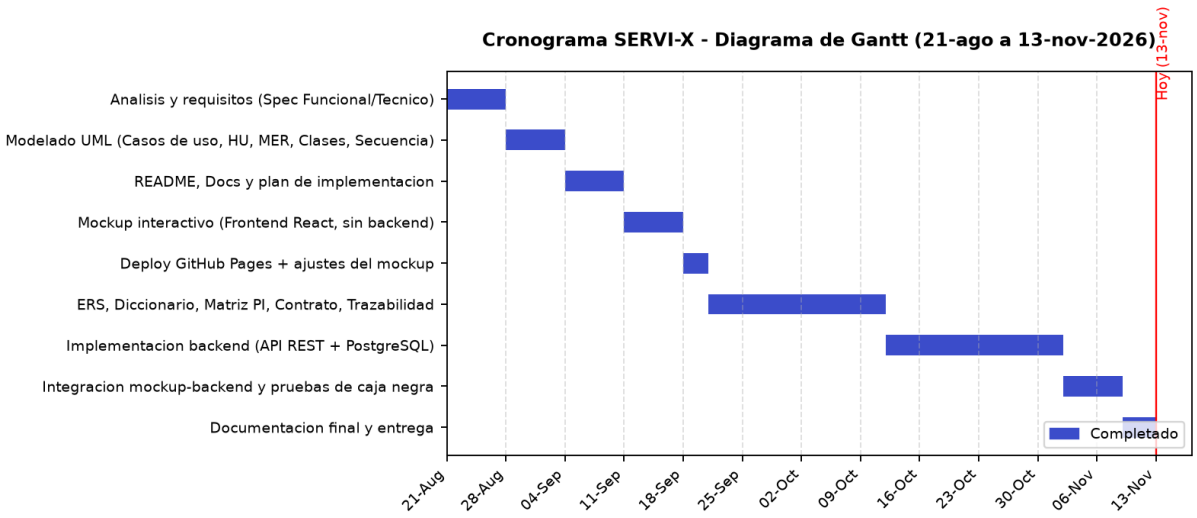


Figura 0. Diagrama de Gantt del proyecto SERVI-X - fases completadas.

Todas las fases del proyecto, desde el analisis de requisitos hasta la implementacion del backend con PostgreSQL, la integracion con el frontend y las pruebas de caja negra, se completaron: la aplicacion completa -frontend, API REST y base de datos- esta funcional y verificada de extremo a extremo (ver seccion 10).

1. Especificacion de Requisitos de Software (ERS - IEEE 830)

1.1 Introduccion

La presente Especificacion de Requisitos de Software (ERS) documenta de manera formal las necesidades funcionales y no funcionales del modulo SERVI-X, siguiendo la estructura recomendada por el estandar IEEE 830. Su proposito es servir como contrato tecnico entre el equipo desarrollador y los interesados del proyecto academico INTEGRAX, garantizando una base verificable para el diseno, la implementacion y las pruebas del sistema.

1.2 Proposito

El proposito de este documento es especificar, sin ambiguedad, el comportamiento esperado del modulo SERVI-X: que debe hacer el sistema (requisitos funcionales), bajo que condiciones de calidad debe operar (requisitos no funcionales), y que restricciones tecnicas, reglas de negocio y criterios de aceptacion enmarcan su desarrollo.

1.3 Alcance

SERVI-X es el modulo encargado de centralizar, clasificar, asignar, atender, responder y cerrar las solicitudes de PQRS de los clientes de la plataforma INTEGRAX, garantizando trazabilidad inmutable de cada actuacion. El sistema ya opera de forma funcional completa: frontend en React, API REST en Node.js/TypeScript y base de datos PostgreSQL propia, verificado mediante las pruebas de la seccion 10.

Aclaracion de alcance: las integraciones salientes hacia CRM-X, SALES-X y LOGISTI-X estan fuera del alcance funcional de esta entrega: se reserva el modelo de datos para ellas (campos cliente_id, pedido_id, despacho_id como llaves foraneas logicas), pero su consumo real via HTTP/REST queda planificado para una integracion posterior dentro del cronograma general de INTEGRAX.

1.4 Descripcion general

1.4.1 Perspectiva del producto: SERVI-X es un sistema web compuesto por un frontend en React (Vite) y una API REST en Node.js/TypeScript (Express), respaldada por una base de datos relacional PostgreSQL. Es uno de los seis microproyectos independientes que conforman INTEGRAX y esta disenado bajo una Arquitectura Orientada a Servicios (SOA) para integrarse posteriormente con los modulos restantes.

1.4.2 Funciones principales: registro y autenticacion de usuarios; radicacion, clasificacion, consulta y seguimiento de PQRS; asignacion de responsables; actualizacion de estados con trazabilidad obligatoria; registro de respuestas oficiales; cierre controlado de casos; generacion de reportes de gestion.

1.4.3 Usuarios del sistema:

- **Cliente:** radica solicitudes, consulta sus propias solicitudes y su historial, y visualiza las respuestas oficiales.
- **Agente de servicio:** clasifica, asigna, gestiona el estado, responde, cierra casos y genera reportes de gestion.
- **Administrador:** ademas de las funciones de Agente, gestiona (crea, edita, desactiva/reactiva) los perfiles operativos del modulo.

1.5 Requisitos Funcionales

Los requerimientos funcionales se numeran de forma alineada uno a uno con los 10 casos de uso (seccion 3) y con la Matriz de Trazabilidad (seccion 9). Los requisitos RF-11 y RF-12 son transversales de integracion empresarial y no tienen un caso de uso dedicado.

ID	Nombre del Requerimiento	Descripcion Detallada	UC
RF-01	Crear usuario	Permite el autorregistro de Clientes, generando un identificador unico (UUID v4) e iniciando su ciclo de vida en el sistema.	UC-01
RF-02	Ingresar	Autentica a Clientes y Agentes, emitiendo un JSON Web Token con el rol del usuario para controlar el acceso.	UC-02
RF-03	Consultar solicitudes	Habilita la visualizacion de un listado de casos con sus estados actuales, con busqueda por numero de solicitud y filtros por fecha, estado y tipo; para un Cliente, filtra automaticamente por su propio identificador.	UC-03
RF-04	Consultar historial de atencion	Muestra la linea de tiempo inmutable de un caso, detallando fechas, responsables y transiciones de estado extraidas de la tabla de auditoria.	UC-04
RF-05	Registrar PQRS	Permite crear una nueva solicitud clasificandola segun su tipologia exacta (Peticion, Queja, Reclamo, Solicitud, Felicitacion u Observacion), con pedido o despacho asociados de forma opcional.	UC-05
RF-06	Asignar responsables	Permite vincular un agente operativo activo (responsable_id) a un caso abierto para gestionar su resolucion.	UC-06
RF-07	Actualizar estado de atencion	Permite modificar la etapa del caso, exigiendo justificacion y registrando la transicion (estado anterior/nuevo) en la bitacora.	UC-07

ID	Nombre del Requerimiento	Descripcion Detallada	UC
RF-08	Generar reportes de gestion	Consolida metricas operativas (cantidad de casos por tipo de solicitud y por estado de atencion) consultando directamente la base de datos aislada.	UC-08
RF-09	Cerrar casos	Finaliza el ciclo de vida de la PQRS unicamente si existe una respuesta oficial previa (RN-03), estampando la fecha de cierre y bloqueando el registro contra modificaciones futuras.	UC-09
RF-10	Registrar respuestas	Habilita la redaccion e insercion de la respuesta formal que se brindara al cliente como solucion a su solicitud.	UC-10
RF-11	Consultar informacion asociada a pedidos y despachos	Permite a SERVI-X consumir APIs de modulos externos (SALES-X, LOGISTI-X) para enriquecer el detalle de un caso. (Integracion planificada para fase posterior; no activa en esta entrega.)	-
RF-12	Exponer servicios para integracion empresarial	Provee endpoints para que otros modulos del ecosistema INTEGRAX u otras plataformas externas puedan radicar o consultar casos.	-

1.6 Requisitos No Funcionales

ID	Categoria	Descripcion Detallada
RNF-01	Disponibilidad	El modulo SERVI-X debe garantizar un tiempo de operatividad minimo del 95%.
RNF-02	Rendimiento	Toda peticion estandar realizada al backend debe procesarse en un tiempo inferior a 3 segundos (3000 ms).
RNF-03	Usabilidad	La interfaz de usuario debe ser intuitiva, adaptandose a distintos tamanos de pantalla.
RNF-04	Seguridad	El sistema debe validar la identidad mediante JWT y restringir el acceso segun los roles (Cliente, Agente, Administrador).
RNF-05	Auditabilidad	Persistencia de una bitacora inmutable (tabla historial_atencion) que impida la sobrescritura del pasado.
RNF-06	Arquitectura	Toda la comunicacion entre el frontend, SERVI-X y los demas modulos debe realizarse bajo los principios REST.
RNF-07	Portabilidad	Las vistas deben renderizarse correctamente en navegadores de escritorio y dispositivos moviles.
RNF-08	Escalabilidad	La base de datos y los servicios deben soportar el incremento transaccional, apoyandose en UUID v4.

1.7 Restricciones

- Lenguaje y entorno de ejecucion: Node.js 20+ con TypeScript.
- Framework web: Express.js.
- Frontend: React con Vite.
- Motor de base de datos: PostgreSQL, exclusivo del modulo (sin tablas compartidas con otros microproyectos).
- Todas las llaves primarias y foraneas deben ser UUID v4; queda prohibido el uso de enteros autoincrementales.
- Nomenclatura de base de datos en snake_case; contratos de API en camelCase.

1.8 Interfaces

- **Interfaz de usuario:** aplicacion web responsiva (portal Cliente, portal Agente y portal Administrador), accesible desde navegador en escritorio y movil.
- **Interfaz de hardware:** ninguna especial; el sistema corre sobre infraestructura de servidor virtualizada/en contenedores.
- **Interfaz de software:** API REST en formato JSON como unico canal de comunicacion entre el frontend y el backend.

1.9 Reglas de Negocio

ID	Regla
RN-01	Todo identificador (caso, usuario, historial) debe ser UUID v4, nunca secuencial.
RN-02	SERVI-X opera sobre su propio repositorio de datos; debe seguir funcionando aunque otros modulos del ecosistema fallen.
RN-03	No se puede cerrar un caso si no existe al menos una respuesta oficial registrada previamente.
RN-04	Todo caso nuevo debe generar simultaneamente su primer registro en la bitacora de auditoria.

1.10 Criterios de Aceptacion

- El sistema permite registrar, autenticar, radicar, asignar, actualizar, responder, cerrar y consultar PQRS cumpliendo los 10 casos de uso definidos - verificado en la seccion 10 con pruebas de caja negra reales.
- Ninguna operacion de cierre se ejecuta sin respuesta oficial previa (RN-03), verificado con prueba de integracion CP-06.

- La tabla historial_atencion rechaza operaciones UPDATE/DELETE a nivel de base de datos (trigger, ver seccion 11).
- Los contratos JSON usan camelCase y las columnas de base de datos usan snake_case de forma consistente.

2. Modelo de Entidad-Relacion

El modelo de datos relacional define la estructura de almacenamiento exclusiva y aislada para el modulo SERVI-X. Para asegurar la escalabilidad dentro del ecosistema de microservicios, el esquema utiliza identificadores universales (UUID v4) como llaves primarias, previniendo colisiones de datos en la red, y aplica estrictamente el estandar snake_case. El modelo garantiza la integridad de la informacion mediante las siguientes entidades principales y sus relaciones de cardinalidad:

Cliente: usuario final que radica solicitudes; se autorregistra (UC-01).

Caso (Entidad Central): Es el nucleo transaccional de la PQRS. Almacena los atributos descriptivos (tipo de solicitud, estado) y las referencias externas (cliente, pedido, despacho).

Historial_Atencion (Trazabilidad y Auditoria): Bitacora inmutable disenada para registrar cualquier transicion o actuacion sobre una solicitud, almacenando el estado anterior, el nuevo y la respuesta dada.

- **Cardinalidad Cliente (1) a Caso (N):** un cliente puede radicar multiples casos a lo largo del tiempo.
- **Cardinalidad Caso (1) a Historial (N):** un solo caso puede acumular multiples actuaciones a lo largo del tiempo, pero cada registro historico pertenece unica y exclusivamente a un caso especifico.

Responsable (Gestion de Actores): Entidad parametrica que administra los datos de los agentes internos (nombre, rol, email, estado).

- **Cardinalidad Responsable (1) a Caso (N):** un agente puede tener asignados multiples casos simultaneamente, mientras que un caso, en un momento dado, solo tiene un unico responsable a cargo.
- **Cardinalidad Responsable (1) a Historial (N):** un agente puede generar multiples registros de actualizacion en el tiempo, permitiendo identificar de forma inequivoca que empleado ejecuto cada accion.

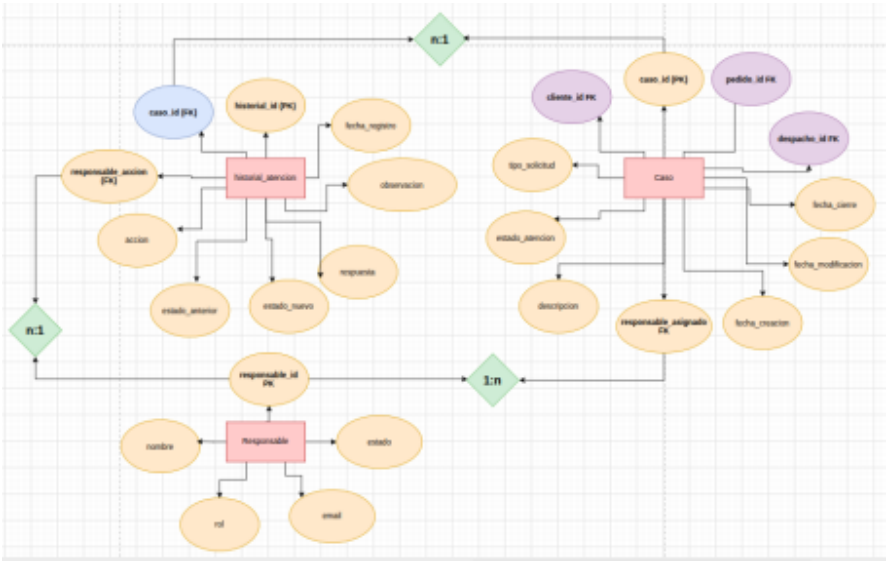


Figura 1. Modelo Entidad-Relacion de SERVI-X.

3. Modelo de Casos de Uso

A continuacion, se define el comportamiento logico, las condiciones y el flujo de los casos de uso representados en el modelo UML para el modulo SERVI-X:

ID	Nombre	Actor	Descripcion y Flujo Principal	Precondicion
UC-01	Crear usuario	Cliente	Registro de nuevos clientes: nombre, tipo de documento, numero de documento, email y contrasena.	Ninguna.
UC-02	Ingresar	Cliente, Agente	Autenticacion mediante usuario y contrasena para obtener acceso segun el rol.	Haber creado un usuario previamente.
UC-03	Consultar solicitudes	Cliente, Agente	Visualizacion del listado de PQRS, con busqueda por numero y filtros por fecha, estado o tipo.	El usuario debe estar autenticado.
UC-04	Consultar historial	Cliente, Agente	Linea de tiempo de un caso, con filtros por fecha, estado y tipo, y vista de detalle.	Conocer el ID del caso y estar autenticado.
UC-05	Registrar PQRS	Cliente, Agente	Creacion de una nueva solicitud: Peticion, Queja, Reclamo, Solicitud, Felicitacion u Observacion, con pedido/despacho asociados opcionalmente.	Estar autenticado.
UC-06	Asignar responsables	Agente	Seleccion de un caso y asignacion de un agente operativo.	El caso debe estar en estado Radicado o Abierto.
UC-07	Actualizar estado	Agente	Modificacion de la etapa del caso en progreso, con trazabilidad.	El caso debe estar asignado a un responsable.
UC-08	Generar reportes	Agente	Consolidacion de metricas filtrando por fecha, estado y tipo de solicitud.	Tener historial registrado en el sistema.
UC-09	Cerrar casos	Agente	Finalizacion del ciclo de vida de la PQRS, unicamente cuando ya existe una respuesta oficial registrada (RN-03).	El caso debe tener una respuesta registrada.
UC-10	Registrar respuestas	Agente	Redaccion de la respuesta formal al cliente, asociada directamente al caso.	El caso debe estar en proceso de investigacion.



Figura 2. Modelo de Casos de Uso de SERVI-X.

3.1 Historias de Usuario

HU-01: Crear usuario

Como Cliente

Quiero registrarme en el sistema SERVI-X

Para poder radicar PQRS y consultar el estado de mis solicitudes.

Criterios de aceptacion:

- El formulario en React debe solicitar: nombre completo, tipo de documento, numero de documento, correo electronico y contrasena.
- El sistema debe validar que el correo electronico no se encuentre previamente registrado.
- La contrasena debe almacenarse encriptada mediante bcrypt antes de persistirse.
- El backend debe generar automaticamente un identificador unico UUID v4 para el nuevo cliente.
- El registro debe persistirse en la base de datos relacional PostgreSQL aislada del modulo.
- La peticion HTTP debe procesarse y retornar un codigo de exito en formato JSON en menos de 3000 ms (RNF-02).

HU-02: Ingresar

Como Cliente o Agente de servicio

Quiero autenticarme en el sistema SERVI-X

Para acceder a las funcionalidades correspondientes segun mi rol operativo.

Criterios de aceptacion:

- El sistema debe validar las credenciales contra la base de datos PostgreSQL aislada del modulo.
- El backend debe generar un JSON Web Token (JWT) al autenticar exitosamente al usuario.
- El payload del JWT debe incluir el rol del usuario para el control de acceso (RNF-04).
- Si las credenciales son incorrectas, la API debe retornar HTTP 401 sin revelar detalles especificos.
- El frontend debe almacenar el token y redirigir al usuario a su panel correspondiente.

HU-03: Consultar solicitudes

Como Cliente o Agente de servicio

Quiero consultar el listado de PQRS registradas

Para visualizar el estado actual de las solicitudes y su informacion general.

Criterios de aceptacion:

- El sistema debe retornar el listado en formato JSON usando camelCase para los contratos de red.
- Si el rol autenticado es Cliente, el backend debe filtrar la consulta para retornar unicamente los registros asociados a su UUID.
- La interfaz debe permitir ingresar el numero de solicitud o buscar por nombre del cliente o por descripcion del caso para busqueda directa.
- El listado debe proporcionar filtros funcionales por fecha, estado de atencion y tipo de solicitud.
- La vista debe ser responsiva (RNF-07).

HU-04: Consultar historial

Como Cliente o Agente de servicio

Quiero visualizar la bitacora completa de un caso especifico

Para auditar la linea de tiempo y todas las transiciones de estado ocurridas.

Criterios de aceptacion:

- El endpoint REST debe exigir un identificador de caso valido (UUID v4) en la ruta.
- El backend debe ejecutar una consulta de solo lectura dirigida exclusivamente a la tabla Historial_Atencion.
- El JSON de respuesta debe incluir literalmente los campos: estadoAnterior, estadoNuevo, accionRealizada, usuarioAccionId y fechaRegistro.
- El frontend debe renderizar los datos como una linea de tiempo ordenada cronologicamente.
- El middleware de autenticacion debe garantizar que un cliente solo pueda leer historiales asociados a sus propios casos.
- La interfaz debe incluir una opcion de "Ver detalles" que expanda la informacion completa de cada evento.

HU-05: Registrar PQRS

Como Cliente o Agente de servicio

Quiero registrar una nueva PQRS en el sistema

Para escalar un requerimiento operativo, queja, reclamo, solicitud, felicitacion u observacion formal.

Criterios de aceptacion:

- El sistema debe generar automaticamente un identificador unico UUID v4 para el caso.
- La interfaz debe permitir seleccionar la tipologia exacta: Peticion, Queja, Reclamo, Solicitud, Felicitacion u Observacion.
- El sistema debe permitir opcionalmente registrar un codigo de pedido o de despacho asociado (texto libre, pendiente de validar via SALES-X/LOGISTI-X cuando esa integracion se habilite).
- El registro inicial debe almacenarse en la base de datos PostgreSQL aislada del modulo.
- Se debe insertar simultaneamente un registro inicial en Historial_Atencion para garantizar la trazabilidad de la radicacion (RN-04).

HU-06: Asignar responsables

Como Agente de servicio

Quiero asignar un empleado responsable a un caso abierto

Para garantizar que la solicitud tenga un encargado para su resolucion operativa.

Criterios de aceptacion:

- El sistema debe consultar la entidad Responsable para listar los agentes activos.
- La interfaz debe permitir elegir el responsable entre los agentes disponibles.
- El backend debe actualizar el atributo de asignacion en la tabla Caso mediante la llave UUID del agente.
- Como el caso solo tiene un campo responsable_id (no una lista), el diseno mismo impide tener mas de un responsable simultaneo: reasignar sobrescribe al anterior.
- El sistema debe registrar el evento de asignacion en la bitacora inmutable de auditoria.
- El acceso a esta funcionalidad debe estar restringido al rol de Agente o Administrador (RNF-04).

HU-07: Actualizar estado

Como Agente de servicio

Quiero actualizar el estado de un caso en progreso

Para reflejar el avance real de la gestion operativa.

Criterios de aceptacion:

- El formulario debe exigir que el agente ingrese una justificacion de texto al modificar el estado.
- El controlador debe capturar el valor del estado anterior y el estado nuevo.
- El sistema debe actualizar el atributo estadoAtencion en la entidad Caso dentro de PostgreSQL.
- Se debe insertar una fila transaccional en Historial_Atencion que documente ambos estados (RNF-05).
- La actualizacion exitosa debe notificar a la interfaz cliente sin requerir recargar la pagina completa.

HU-08: Generar reportes

Como Agente de servicio

Quiero generar reportes de gestion del modulo SERVI-X

Para consolidar metricas operativas que apoyen la toma de decisiones.

Criterios de aceptacion:

- El backend debe exponer un endpoint REST especifico para la consolidacion de metricas de negocio.
- El sistema debe ejecutar consultas de agrupacion nativas en PostgreSQL para calcular el volumen total.
- La interfaz debe permitir generar reportes filtrando por rango de fechas.
- Las metricas retornadas deben agrupar la cantidad de casos por estadoAtencion y por tipoSolicitud.
- El payload JSON debe estar formateado para ser renderizado directamente en la interfaz web sin procesamiento adicional.

HU-09: Cerrar casos

Como Agente de servicio

Quiero cerrar un caso que ya tiene respuesta registrada

Para finalizar el ciclo de vida de la PQRS y bloquear modificaciones futuras.

Criterios de aceptacion:

- El backend debe rechazar la solicitud de cierre si el caso no tiene al menos una respuesta oficial previa asociada (RN-03).
- El flujo se realiza desde la vista de detalle del caso ya abierto: el agente no debe reingresar el numero de caso (ya identificado por la pantalla en la que esta) ni una descripcion de cierre adicional - la justificacion de la resolucion es, precisamente, la Respuesta Oficial ya exigida por RN-03; pedir una segunda descripcion duplicaria ese mismo dato.
- El sistema debe estampar la fecha y hora exacta del cierre en la entidad principal del caso (fechaCierre).
- El sistema debe bloquear cualquier operacion de actualizacion posterior sobre ese registro de caso.
- Se debe generar una traza final en Historial_Atencion documentando el cierre.
- El estado actualizado debe reflejarse inmediatamente en el modulo de generacion de reportes.

HU-10: Registrar respuestas

Como Agente de servicio

Quiero redactar una respuesta formal dentro del caso

Para brindar una solucion oficial al requerimiento del cliente.

Criterios de aceptacion:

- El sistema debe habilitar un area de texto extenso en el panel operativo para redactar la respuesta.
- El flujo se realiza desde la vista de detalle del caso: el agente solo ingresa la descripcion de la respuesta; el correo electronico del cliente no se vuelve a pedir porque el caso ya tiene asociado su cliente_id, y de ahi el sistema resuelve el correo automaticamente - evita error humano de tipeo que enviaria la respuesta a un correo incorrecto.
- La respuesta ingresada debe quedar asociada al caso correspondiente.
- El texto oficial debe persistirse como un evento inmutable dentro del historial (RNF-05).
- La respuesta debe quedar disponible en modo lectura para el cliente en su panel de consulta.
- El sistema debe validar que exista al menos una respuesta oficial registrada antes de habilitar el cierre del caso.

4. Modelo de Clases y Estructura de Objetos

El diagrama de clases ilustra la vista estatica del backend para el modulo SERVI-X. Se adopto camelCase para todos los atributos y metodos (facilita la serializacion JSON) y encapsulamiento estricto (atributos privados, mutacion solo mediante metodos publicos).

- **Clase Caso:** objeto central de la logica de PQRS. Gestiona tipoSolicitud y estadoAtencion; metodos actualizarEstado(nuevoEstado) y cerrarCaso().
- **Clase HistorialAtencion:** objeto transaccional de auditoria; metodo registrarActualizacion() garantiza traza inmutable.
- **Clase Responsable:** modela al agente operativo; metodos administrativos como actualizarResponsable().
- **Clase Cliente:** modela al usuario final que radica PQRS; se crea via registrarCliente().

Asociaciones y multiplicidad: Caso 1-* HistorialAtencion; Responsable 1-* Caso; Responsable 1-* HistorialAtencion (como autor de los eventos); Cliente 1-* Caso.

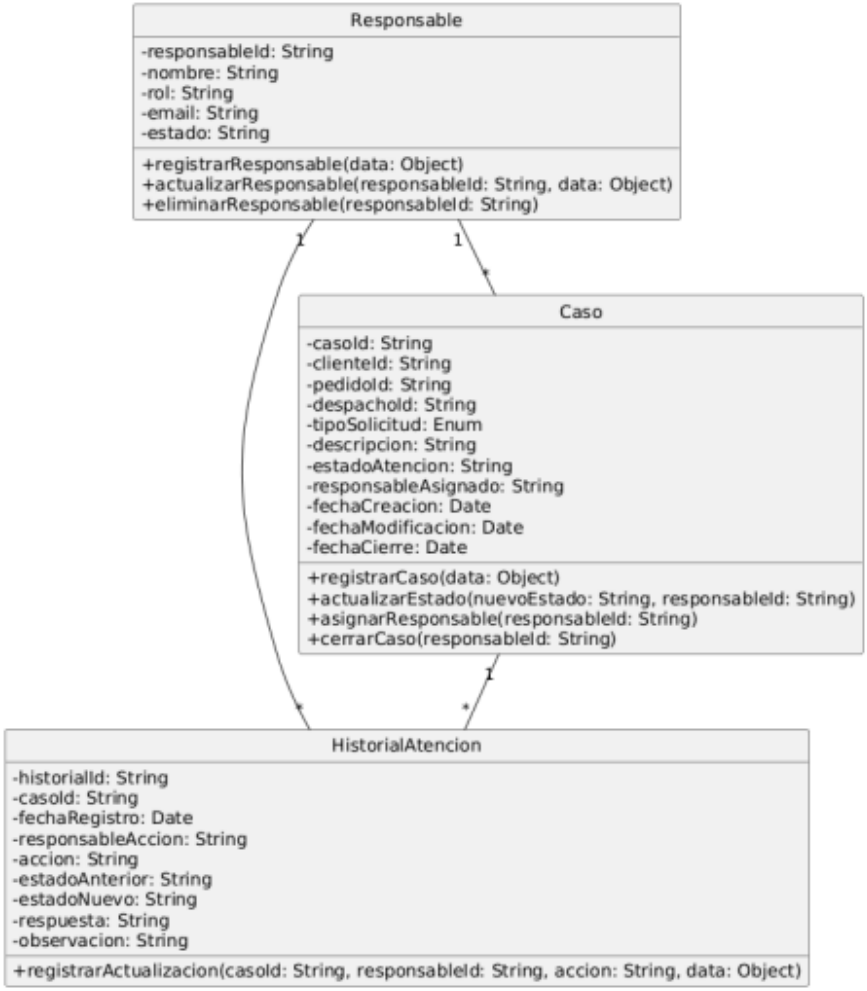


Figura 3. Diagrama de Clases de SERVI-X.

4.1 Diccionario de Datos

Entidad caso

Campo	Tipo	Long.	PK	FK	Oblig.	Descripcion y Regla de Negocio
caso_id	UUID	36	Si	No	Si	Identificador universal unico generado (UUID v4) al radicar.
cliente_id	UUID	36	No	Si (cliente)	Si	Llave foranea hacia la tabla cliente; se resuelve automaticamente desde la sesion del Cliente autenticado.
pedido_id	VARCHAR	100	No	Logica (SALES-X)	No	Codigo de pedido asociado, texto libre; integracion con SALES-X planificada.
despacho_id	VARCHAR	100	No	Logica (LOGISTI-X)	No	Codigo de despacho asociado, texto libre; integracion con LOGISTI-X planificada.
tipo_solicitud	VARCHAR	50	No	No	Si	Enumeracion: 'Peticion', 'Queja', 'Reclamo', 'Solicitud', 'Felicitacion', 'Observacion'.
estado_atencion	VARCHAR	50	No	No	Si	Estado actual (ej. 'Radicado', 'En Proceso', 'Cerrado').
descripcion	TEXT	Variable	No	No	Si	Detalle textual de la necesidad expresada por el cliente.
fecha_creacion	TIMESTAMP	N/A	No	No	Si	Fecha y hora exacta de la radicacion (NOW()).
fecha_cierre	TIMESTAMP	N/A	No	No	No	Se estampa unicamente cuando el caso finaliza su ciclo de vida.
responsable_id	UUID	36	No	Si (responsable)	No	ID del Agente asignado.

Entidad cliente

Campo	Tipo	Long.	PK	FK	Oblig.	Descripcion y Regla de Negocio
cliente_id	UUID	36	Si	No	Si	Identificador del cliente (UUID v4).
nombre_completo	VARCHAR	150	No	No	Si	Nombre y apellidos del cliente.
tipo_documento	VARCHAR	50	No	No	Si	Tipo de documento de identidad.
numero_documento	VARCHAR	50	No	No	Si	Numero de documento de identidad.
email	VARCHAR	100	No	No	Si (unico)	Correo electronico y credencial de acceso.
password_hash	VARCHAR	255	No	No	Si	Contrasena encriptada con bcrypt (salt rounds 10).
fecha_creacion	TIMESTAMP	N/A	No	No	Si	Fecha de autorregistro.

Entidad historial_atencion

Tabla estrictamente inmutable (solo soporta sentencias INSERT; un trigger de PostgreSQL rechaza UPDATE y DELETE - ver seccion 11).

Campo	Tipo	Long.	PK	FK	Oblig.	Descripcion y Regla de Negocio
historial_id	UUID	36	Si	No	Si	Identificador unico del evento registrado (UUID v4).
caso_id	UUID	36	No	Si (caso)	Si	Relacion obligatoria hacia la tabla caso.
estado_anterior	VARCHAR	50	No	No	No	Estado previo del caso (NULL unicamente en el primer registro).

Campo	Tipo	Long.	PK	FK	Oblig.	Descripcion y Regla de Negocio
estado_nuevo	VARCHAR	50	No	No	Si	Nuevo estado asignado tras la accion del usuario.
accion_realizada	TEXT	Variable	No	No	Si	Justificacion obligatoria o respuesta oficial brindada.
usuario_accion_id	UUID	36	No	Si (resp./cliente)	Si	ID del usuario (Cliente o Agente) que ejecuto el cambio.
accion	VARCHAR	50	No	No	Si	Tipo de evento: Creacion, Asignacion, CambioEstado, RespuestaOficial, Cierre.
fecha_registro	TIMESTAMP	N/A	No	No	Si	Estampa de tiempo exacta de la transaccion.

Entidad responsable

Campo	Tipo	Long.	PK	FK	Oblig.	Descripcion y Regla de Negocio
responsable_id	UUID	36	Si	No	Si	Identificador del agente (UUID v4).
nombre_completo	VARCHAR	150	No	No	Si	Nombre y apellidos del funcionario.
correo_corporativo	VARCHAR	100	No	No	Si (unico)	Medio de contacto y credencial de acceso.
password_hash	VARCHAR	255	No	No	Si	Contrasena encriptada con bcrypt (salt rounds 10).
rol_operativo	ENUM	-	No	No	Si	'Agente' o 'Administrador'.
estado_activo	BOOLEAN	1	No	No	Si	Indica si el agente esta habilitado (TRUE) para recibir asignaciones.

5. Diagrama de Secuencia

El modelo de secuencia expone el comportamiento dinamico del sistema y el flujo temporal de los mensajes intercambiados entre los distintos componentes del modulo SERVI-X. El diagrama ilustra la interaccion interna entre el Frontend, la API REST, la capa de Servicio de Negocio y la Base de Datos PostgreSQL aislada, sin aun conectarse con los servicios externos del ecosistema INTEGRAX.

- **Autenticacion y Control de Acceso:** registro con validacion de unicidad de correo y hash de contrasena; ingreso con validacion de credenciales y emision de JWT con el rol embebido (RNF-04).
- **Transaccionalidad y Registro de Auditoria:** cada accion relevante ejecuta la insercion simultanea en caso e historial_atencion, garantizando trazabilidad e inmutabilidad (RNF-05).
- **Gestion del Ciclo de Vida de la PQRS:** asignacion, clasificacion, actualizacion de estado con justificacion, respuesta oficial y cierre con validacion condicional de RN-03.
- **Consultas y Reporteria:** listados con filtros dinamicos, consulta de la linea de tiempo, y reportes agregados (GROUP BY).
- **Estandares de Interoperabilidad:** respuestas en JSON camelCase; persistencia en PostgreSQL snake_case.

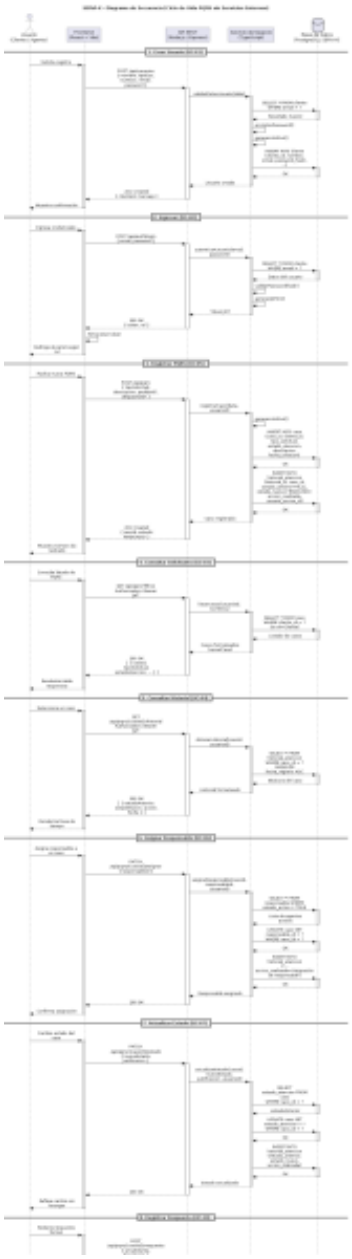


Figura 4. Diagrama de Secuencia - Ciclo de vida de una PQRS (sin servicios externos).

6. Arquitectura

6.1 Ficha tecnica de SOFTWARE

Campo	Descripcion
Nombre del sistema / proyecto	Modulo SERVI-X (Ecosistema INTEGRAX)
Lenguaje(s) de programacion	TypeScript y entorno de Node. Base de datos relacional PostgreSQL
Framework(s) / librerias principales	Express.js para la API REST, Prisma ORM, y React con Vite para el frontend
Motor de base de datos	PostgreSQL 16
Sistema operativo compatible	Multiplataforma 100% web, accesible desde cualquier navegador moderno.
Herramientas de desarrollo	Visual Studio Code, Git/GitHub, Docker, Jest/Supertest.
Tipo de licenciamiento	Propietario / Academico (ver secciones 7 y 8).
Requerimientos funcionales principales	Gestion del ciclo de vida de PQRS, registro historico inmutable y generacion de reportes.
Modulos / servicios de integracion	Consumo planificado (no activo en esta fase) de CRM-X, SALES-X y LOGISTI-X.

6.2 Ficha tecnica de HARDWARE

Campo	Descripcion
Tipo de servidor	Servidor en la nube virtualizado mediante contenedores Docker (Railway planificado).
Procesador	Minimo 2 vCPU.
Memoria RAM	Minima 2GB / Recomendada 4GB.
Almacenamiento	Minimo 20GB SSD.
Requerimientos de red	Ancho de banda que garantice respuestas en menos de 3 segundos.
Dispositivos cliente compatibles	Multiplataforma 100% web.
Proveedor de infraestructura	Despliegue planificado en la nube (Railway), con integracion continua desde GitHub.
Mecanismos de respaldo	Backups automaticos planificados via GitHub Actions.

6.3 Ficha tecnica de ARQUITECTURA

Campo	Descripcion
Tipo de arquitectura	Arquitectura Orientada a Servicios (SOA) con persistencia de datos relacional y aislada.
Patron de integracion	Comunicacion sincrona mediante API REST (consumo externo planificado, no activo en esta fase).
Componentes / capas	Capa de Presentacion (React), Capa de Logica/API (Express + Prisma), Capa de Datos (PostgreSQL aislada).
Protocolos de comunicacion	HTTP en formato JSON, camelCase.
Mecanismo de seguridad	JWT con roles (Cliente, Agente, Administrador).
Estrategia de escalabilidad	UUID v4 en todas las llaves.
Servicios o APIs externas	Planificadas: /api/clientes (CRM-X), /api/pedidos (SALES-X), /api/despachos (LOGISTI-X). No activas en esta entrega.
Estrategia de despliegue	Dockerfile para el backend; docker-compose para backend + PostgreSQL local.

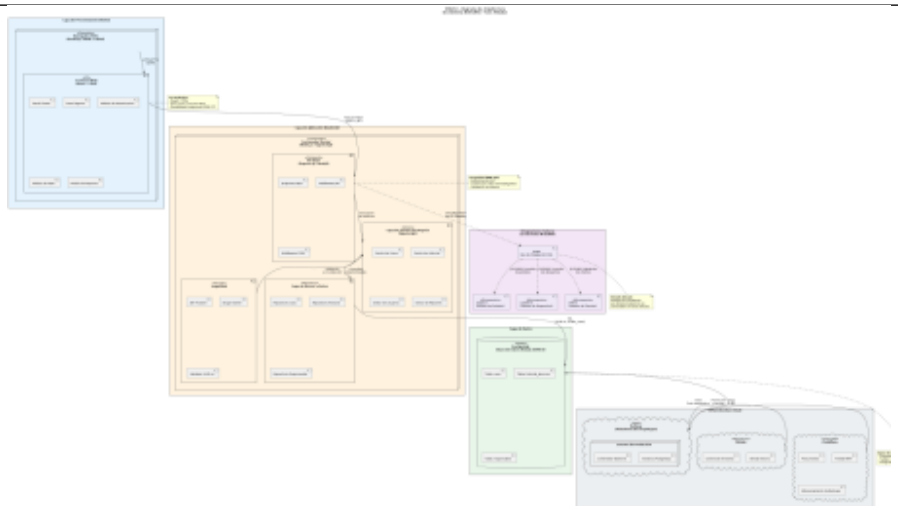


Figura 5. Diagrama de Arquitectura de SERVI-X dentro del ecosistema INTEGRAX.

7. Matriz de Propiedad Intelectual

La siguiente matriz identifica cada componente de software de terceros utilizado en el modulo SERVI-X, su procedencia, su tipo de licencia y el uso especifico que se le da dentro del proyecto.

Elemento	Procedencia	Licencia	Uso en el proyecto
React	Meta Platforms, Inc. / comunidad open source	MIT	Libreria base del frontend (SPA, componentes de UI).
Vite	Evan You / comunidad open source	MIT	Bundler y servidor de desarrollo del frontend.
react-router-dom	Remix Software / comunidad open source	MIT	Enrutamiento de vistas en el frontend.
Node.js	OpenJS Foundation	MIT	Entorno de ejecucion del backend.
Express.js	OpenJS Foundation / comunidad open source	MIT	Framework web para exponer la API REST.
TypeScript	Microsoft Corporation	Apache License 2.0	Lenguaje tipado usado en todo el backend.
PostgreSQL	PostgreSQL Global Development Group	Licencia PostgreSQL (estilo MIT/BSD)	Motor de base de datos relacional del modulo.
Prisma ORM	Prisma Data, Inc. / comunidad open source	Apache License 2.0	Mapeo objeto-relacional y migraciones de base de datos.
bcrypt	Comunidad open source	MIT	Hashing de contrasenas.
jsonwebtoken	Auth0 / comunidad open source	MIT	Emision y verificacion de JSON Web Tokens.
Jest	OpenJS Foundation / Meta	MIT	Framework de pruebas unitarias e integracion del backend.
Supertest	Comunidad open source	MIT	Pruebas de integracion sobre los endpoints REST.
Docker	Docker, Inc.	Apache License 2.0 (Docker Engine)	Contenerizacion del backend y la base de datos para despliegue.

Elemento	Procedencia	Licencia	Uso en el proyecto
Codigo propio del equipo SERVI-X	Equipo desarrollador (Jose Alvarez Marin)	Derechos reservados - trabajo academico de la Corporacion Universitaria de Sabaneta	Logica de negocio, modelos de dominio, controladores, vistas y configuracion especifica del modulo SERVI-X.

8. Contrato de Licencia de Software

CONTRATO DE LICENCIAMIENTO Y PRESTACION DE DESARROLLO DE SOFTWARE

Identificacion del software: el presente contrato tiene por objeto el software denominado SERVI-X (modulo de gestion de PQRS del ecosistema INTEGRAX), en su version documentada en este mismo documento tecnico (modelo de datos, diccionario de datos, modelo de clases y arquitectura descritos en las secciones 2 a 6).

Entre los suscritos, a saber: de una parte, INTEGRAX (en adelante, "EL CLIENTE"), plataforma empresarial de integracion constituida por los seis microproyectos academicos de la Electiva Profesional II - Integracion de Aplicaciones y Sistemas de la Corporacion Universitaria de Sabaneta; y de otra parte, el equipo desarrollador del modulo SERVI-X, representado por Jose Alvarez Marin (en adelante, "EL DESARROLLADOR"), se celebra el presente contrato academico de licenciamiento de software, regido por las siguientes clausulas:

CLAUSULA PRIMERA - Identificacion de las partes. EL DESARROLLADOR es el autor y titular original del software denominado SERVI-X, modulo de gestion de PQRS. EL CLIENTE es la plataforma integradora INTEGRAX, que recibe el derecho de uso del modulo para su incorporacion dentro del ecosistema empresarial simulado.

CLAUSULA SEGUNDA - Objeto del contrato. EL DESARROLLADOR concede a EL CLIENTE una licencia de uso sobre el software SERVI-X, en su version entregada al cierre de cada hito academico (semanas 1 a 5), para su operacion, integracion y evaluacion dentro de la plataforma INTEGRAX.

CLAUSULA TERCERA - Titularidad. EL DESARROLLADOR conserva en todo momento los derechos morales y la titularidad intelectual sobre el codigo fuente original de SERVI-X, de conformidad con la Ley 23 de 1982 sobre derechos de autor. La presente licencia no implica cesion ni transferencia de la propiedad intelectual, sino unicamente la concesion de derechos de uso definidos en este contrato.

CLAUSULA CUARTA - Derechos de uso. EL CLIENTE podra instalar, ejecutar, consultar e integrar el software SERVI-X con los demas modulos del ecosistema INTEGRAX (CRM-X, SALES-X, FINAN-X, LOGISTI-X, INVENTA-X), exclusivamente dentro del marco academico de la asignatura Integracion de Aplicaciones y Sistemas.

CLAUSULA QUINTA - Duracion. El presente contrato tendra vigencia desde la fecha de su firma hasta la finalizacion del periodo academico correspondiente (18 de septiembre de 2026), pudiendo prorrogarse por mutuo acuerdo para fases posteriores de integracion real.

CLAUSULA SEXTA - Territorio. La licencia se concede para uso dentro del territorio academico de la Corporacion Universitaria de Sabaneta y, de manera extendida, para cualquier entorno de

demostracion o evaluacion docente asociado al proyecto INTEGRAX.

CLAUSULA SEPTIMA - Exclusividad. La presente licencia es de caracter no exclusivo. EL DESARROLLADOR conserva el derecho de reutilizar, adaptar o presentar el codigo fuente de SERVI-X en otros contextos academicos o profesionales, siempre que no comprometa la confidencialidad de datos propios de EL CLIENTE.

CLAUSULA OCTAVA - Restricciones. EL CLIENTE no podra sublicenciar, vender, ni distribuir comercialmente el software SERVI-X a terceros ajenos al ecosistema INTEGRAX sin autorizacion expresa y escrita de EL DESARROLLADOR.

CLAUSULA NOVENA - Uso comercial. Cualquier explotacion comercial del software, fuera del contexto academico aqui descrito, requerira la negociacion de un nuevo acuerdo de licenciamiento entre las partes, incluyendo la correspondiente contraprestacion economica.

CLAUSULA DECIMA - Propiedad intelectual. La propiedad intelectual sobre SERVI-X se distribuye de la siguiente manera: (i) el codigo propio desarrollado por EL DESARROLLADOR permanece bajo sus derechos reservados conforme a la clausula tercera; (ii) los componentes de terceros incorporados mantienen la titularidad de sus respectivos autores y se rigen por sus licencias originales. EL CLIENTE no adquiere derecho de propiedad alguno sobre ninguno de los dos, unicamente el derecho de uso aqui concedido.

CLAUSULA DECIMA PRIMERA - Componentes de terceros. EL CLIENTE reconoce que SERVI-X incorpora componentes de software de terceros bajo licencias de codigo abierto (MIT, Apache License 2.0 y Licencia PostgreSQL, entre otras), segun se detalla en la Matriz de Propiedad Intelectual (seccion 7 de este documento), y se compromete a respetar los terminos de dichas licencias.

CLAUSULA DECIMA SEGUNDA - Distribucion y modificacion. EL CLIENTE no podra modificar, descompilar, ni generar obras derivadas del codigo fuente de SERVI-X sin autorizacion previa y escrita de EL DESARROLLADOR. Cualquier distribucion del software a terceros dentro del ecosistema INTEGRAX debera conservar los avisos de autoria y de licenciamiento originales.

CLAUSULA DECIMA TERCERA - Soporte y mantenimiento. EL DESARROLLADOR prestara soporte y mantenimiento correctivo sobre el software durante la vigencia del periodo academico (clausula quinta), limitado a la correccion de errores detectados en las pruebas de integracion (seccion 10 de este documento). Cualquier mantenimiento evolutivo o soporte posterior a la finalizacion del periodo academico requerira un acuerdo independiente entre las partes.

CLAUSULA DECIMA CUARTA - Terminacion. El presente contrato podra darse por terminado de forma anticipada por incumplimiento de cualquiera de las partes, o de forma ordinaria al concluir el periodo academico pactado, sin perjuicio de la conservacion de los derechos morales

de EL DESARROLLADOR.

En constancia de lo anterior, se firma el presente contrato academico el 12 de noviembre de 2026.

_____

EL DESARROLLADOR - Jose Alvarez Marin

_____

EL CLIENTE - INTEGRAX (representado academicamente por el docente evaluador)

9. Matriz de Trazabilidad

La siguiente matriz demuestra la cadena logica de trazabilidad entre cada requisito funcional, su caso de uso de origen, la clase y entidad del modelo de datos que lo materializan, el endpoint REST que lo expone y la prueba de integracion disenada para verificarlo. La numeracion RF coincide exactamente con la de la seccion 1.5.

RF	Caso de Uso (UC)	Clase	Entidad (MER)	Funcionalidad / API (Endpoint)	Prueba
RF-01	UC-01 Crear usuario	Cliente	cliente	POST /api/v1/usuarios	CP-01
RF-02	UC-02 Ingresar	Cliente / Responsable	cliente / responsable	POST /api/v1/auth/login	CP-02, CP-03
RF-03	UC-03 Consultar solicitudes	Caso	caso	GET /api/v1/casos	CP-10
RF-04	UC-04 Consultar historial	HistorialAtencion	historial_atencion	GET /api/v1/casos/{id}/historial	CP-10
RF-05	UC-05 Registrar PQRS	Caso	caso	POST /api/v1/casos	CP-04
RF-06	UC-06 Asignar responsables	Caso	caso	PATCH /api/v1/casos/{id}/asignacion	CP-05
RF-07	UC-07 Actualizar estado	Caso	caso	PATCH /api/v1/casos/{id}/estado	CP-07
RF-08	UC-08 Generar reportes	Caso	caso	GET /api/v1/reportes/resumen	CP-09
RF-09	UC-09 Cerrar casos	Caso	caso	PATCH /api/v1/casos/{id}/cierre	CP-06, CP-08
RF-10	UC-10 Registrar respuestas	HistorialAtencion	historial_atencion	POST /api/v1/casos/{id}/respuesta	CP-07

10. Pruebas de Integracion

Las siguientes diez pruebas son de caja negra: se ejecutaron interactuando unicamente con la interfaz de usuario de la aplicacion real (frontend React en http://localhost:5173, backend Express en http://localhost:3000, PostgreSQL 16), sin inspeccionar ni depender del codigo interno, demostrando que los tres componentes (frontend, API y base de datos) se integran y funcionan correctamente de extremo a extremo:

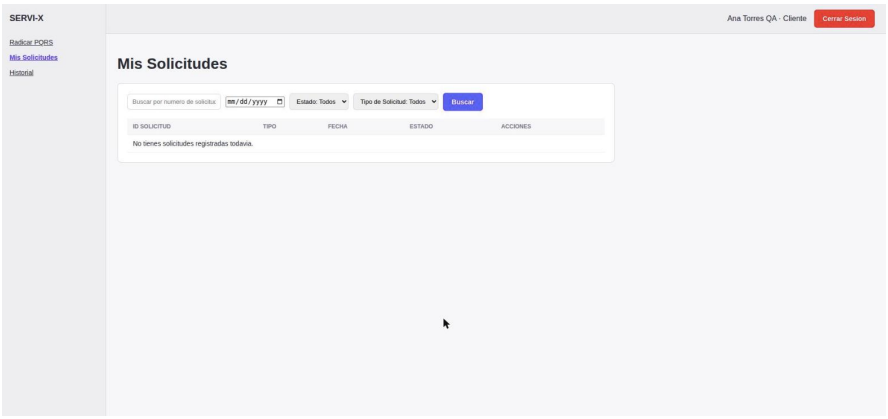
Usuario -> Frontend -> API -> Backend -> Base de datos -> Respuesta -> Frontend -> Usuario

Cada prueba se diseno con el formato estandar de caso de prueba (objetivo, precondicion, datos de entrada, pasos, resultado esperado, resultado obtenido, estado PASS/FAIL, evidencia fotografica, error encontrado y correccion realizada) y fue ejecutada el 12 de noviembre de 2026.

CP-01 - Crear usuario (Cliente)

Objetivo / RF - UC	RF-01 / UC-01
Precondicion	Ninguna. El correo no debe existir previamente en la base de datos.
Datos de entrada	Nombre: Ana Torres QA; Documento: Cedula de Ciudadania 1122334455; Email: ana.torres.qa@example.com; Contrasena: Clave123*
Pasos ejecutados	1) Navegar a /#/registro. 2) Completar el formulario de registro. 3) Clic en "Acceder".
Resultado esperado	El backend crea el Cliente en PostgreSQL con contrasena hasheada (bcrypt), genera un UUID v4 y un JWT, y el frontend redirige automaticamente a "Mis Solicitudes" ya autenticado.
Resultado obtenido	Igual al esperado: el usuario "Ana Torres QA - Cliente" quedo autenticado en la pantalla "Mis Solicitudes" (ver evidencia).
Estado	PASS
Error encontrado	Ninguno.
Correccion realizada	No aplica.

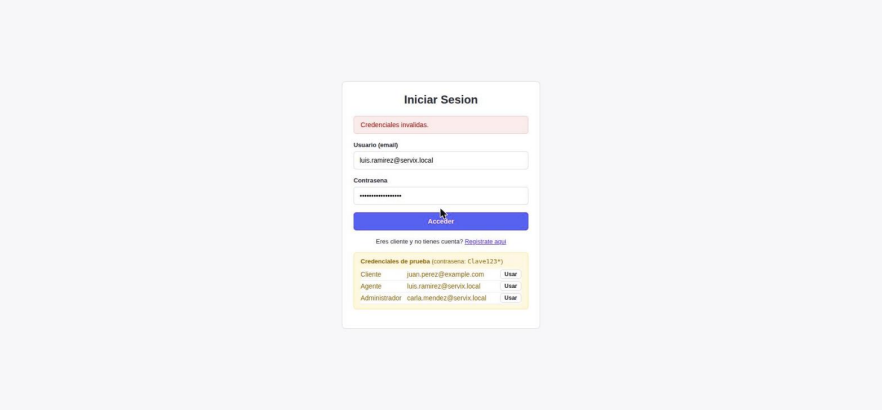
Evidencia (captura de pantalla real):



CP-02 - Rechazo de credenciales invalidas (negativo)

Objetivo / RF - UC	RF-02 / UC-02
Precondicion	El responsable luis.ramirez@servix.local existe en el sistema (seed).
Datos de entrada	Email: luis.ramirez@servix.local; Contrasena: ClaveIncorrecta999 (incorrecta).
Pasos ejecutados	1) Navegar a /#/login. 2) Ingresar el email correcto con contrasena incorrecta. 3) Clic en "Acceder".
Resultado esperado	La API retorna HTTP 401 con el mensaje "Credenciales invalidas.", sin revelar si el email existe, y el frontend no inicia sesion.
Resultado obtenido	Igual al esperado: se mostro el banner de error "Credenciales invalidas." y la sesion no se inicio (ver evidencia).
Estado	PASS
Error encontrado	Ninguno.
Correccion realizada	No aplica.

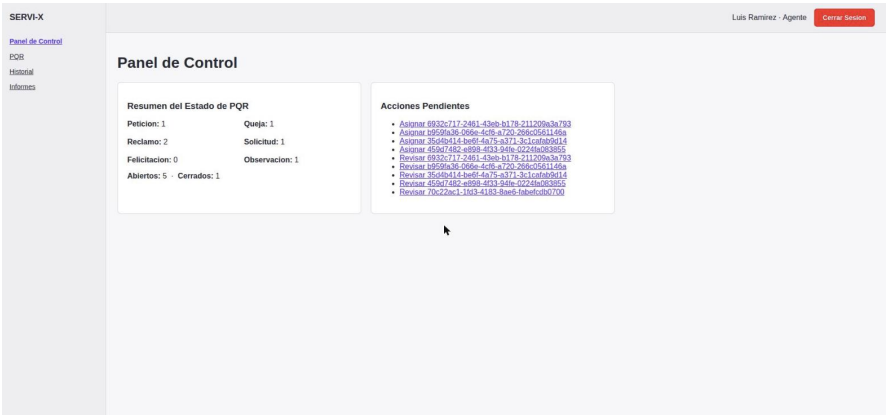
Evidencia (captura de pantalla real):



CP-03 - Ingreso valido de Agente y Panel de Control

Objetivo / RF - UC	RF-02 / UC-02
Precondicion	El responsable luis.ramirez@servix.local existe y esta activo.
Datos de entrada	Email: luis.ramirez@servix.local; Contrasena: Clave123*.
Pasos ejecutados	1) Navegar a /#/login. 2) Usar el boton "Usar" de la fila Agente. 3) Clic en "Acceder".
Resultado esperado	Autenticacion exitosa; redireccion al Panel de Control con el resumen de casos agrupado por los 6 tipos de solicitud definidos en UC-05 y el conteo de abiertos/cerrados.
Resultado obtenido	Igual al esperado. El panel mostro "Luis Ramirez - Agente" y el resumen con las 6 categorias (Peticion, Queja, Reclamo, Solicitud, Felicitacion, Observacion).
Estado	PASS
Error encontrado	Ninguno.
Correccion realizada	No aplica.

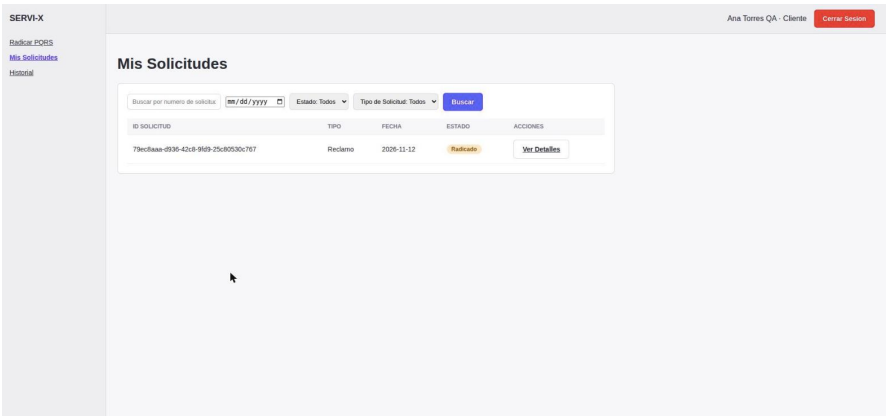
Evidencia (captura de pantalla real):



CP-04 - Registrar PQRS

Objetivo / RF - UC	RF-05 / UC-05
Precondicion	Cliente autenticado (Ana Torres QA, de CP-01).
Datos de entrada	Tipo: Reclamo; Descripcion: "El producto llego con el empaque roto, solicito reemplazo."; Numero de Pedido asociado: SALES-X-QA-01 (texto libre, no UUID).
Pasos ejecutados	1) Menu "Radicar PQRS". 2) Completar tipo, descripcion y pedido asociado. 3) Clic en "Enviar PQRS".
Resultado esperado	Se crea el caso en PostgreSQL con estado "Radicado" y un registro inicial automatico en el historial (RN-04); el campo pedido asociado acepta texto libre sin exigir formato UUID.
Resultado obtenido	Igual al esperado: el caso quedo visible en "Mis Solicitudes" con estado "Radicado".
Estado	PASS
Error encontrado	Ninguno.
Correccion realizada	No aplica.

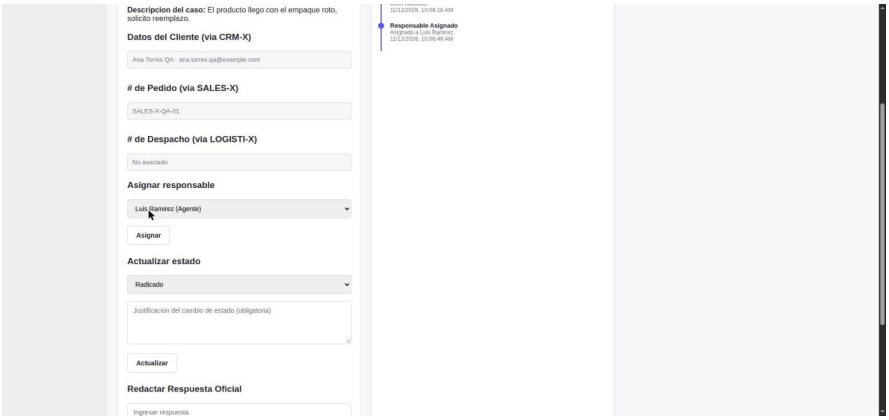
Evidencia (captura de pantalla real):



CP-05 - Asignar responsable

Objetivo / RF - UC	RF-06 / UC-06
Precondicion	Caso de CP-04 en estado Radicado; Agente autenticado.
Datos de entrada	Responsable seleccionado: Luis Ramirez (Agente).
Pasos ejecutados	1) Como Agente, abrir "PQR". 2) Abrir el detalle del caso de Ana Torres QA. 3) Seleccionar responsable y clic en "Asignar".
Resultado esperado	El caso queda asignado a Luis Ramirez; se registra el evento "Responsable Asignado" en la linea de tiempo del historial.
Resultado obtenido	Igual al esperado: el timeline del caso muestra "Responsable Asignado - Asignado a Luis Ramirez." con fecha y hora exactas.
Estado	PASS
Error encontrado	Ninguno.
Correccion realizada	No aplica.

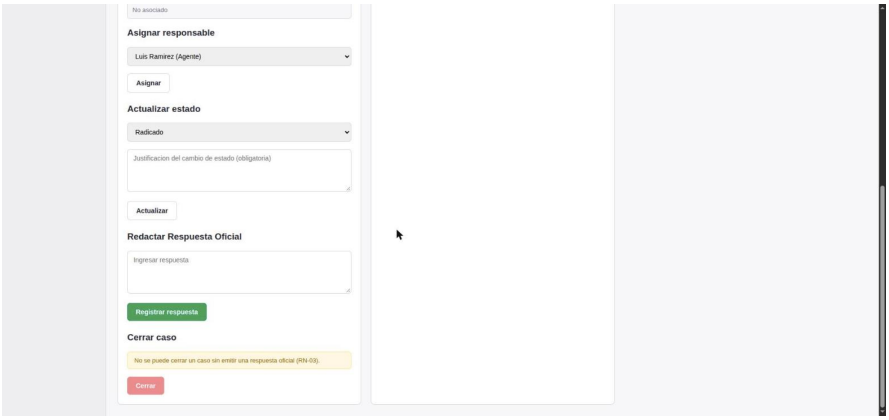
Evidencia (captura de pantalla real):



CP-06 - Bloqueo de cierre sin respuesta oficial (negativo, RN-03)

Objetivo / RF - UC	RF-09 / UC-09 / RN-03
Precondicion	Caso asignado, sin ningun evento "RespuestaOficial" en su historial.
Datos de entrada	Ninguno (solo se intenta la accion de cierre).
Pasos ejecutados	1) En el detalle del caso, intentar hacer clic en "Cerrar".
Resultado esperado	El boton "Cerrar" aparece deshabilitado y se muestra el mensaje "No se puede cerrar un caso sin emitir una respuesta oficial (RN-03)."; la API rechazaria la operacion con HTTP 400 si se forzara.
Resultado obtenido	Igual al esperado: el mensaje de bloqueo se mostro y el boton "Cerrar" quedo deshabilitado (rojo claro).
Estado	PASS
Error encontrado	Ninguno.
Correccion realizada	No aplica.

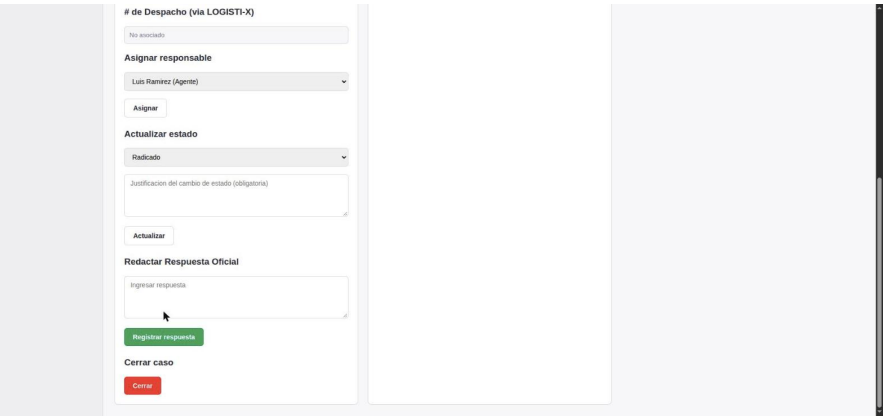
Evidencia (captura de pantalla real):



CP-07 - Registrar respuesta oficial

Objetivo / RF - UC	RF-10 / UC-10
Precondicion	Caso asignado (continuacion de CP-06).
Datos de entrada	Respuesta: "Hemos revisado su reclamo y procederemos a enviar un reemplazo del producto sin costo adicional."
Pasos ejecutados	1) Escribir el texto en "Redactar Respuesta Oficial". 2) Clic en "Registrar respuesta".
Resultado esperado	Se inserta el evento "RespuestaOficial" en el historial, sin pedir el correo del cliente (se resuelve automaticamente desde el caso); el boton "Cerrar" se habilita.
Resultado obtenido	Igual al esperado: tras guardar, el boton "Cerrar" cambio a color rojo solido (habilitado).
Estado	PASS
Error encontrado	Ninguno.
Correccion realizada	No aplica.

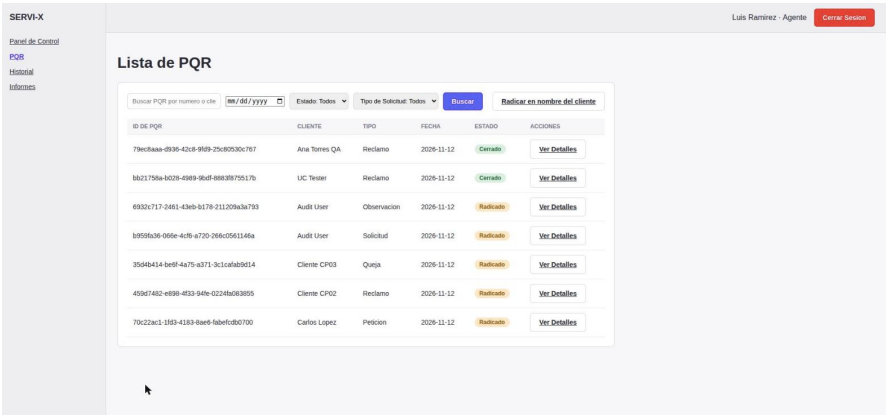
Evidencia (captura de pantalla real):



CP-08 - Cierre exitoso de caso

Objetivo / RF - UC	RF-09 / UC-09
Precondicion	Caso con respuesta oficial registrada (CP-07).
Datos de entrada	Ninguno (solo se ejecuta la accion de cierre).
Pasos ejecutados	1) Clic en "Cerrar".
Resultado esperado	El caso cambia a estado "Cerrado", se estampa fechaCierre, y queda bloqueado contra modificaciones futuras.
Resultado obtenido	Igual al esperado: en la lista de PQR, el caso de Ana Torres QA aparece con estado "Cerrado" (verde).
Estado	PASS
Error encontrado	Ninguno.
Correccion realizada	No aplica.

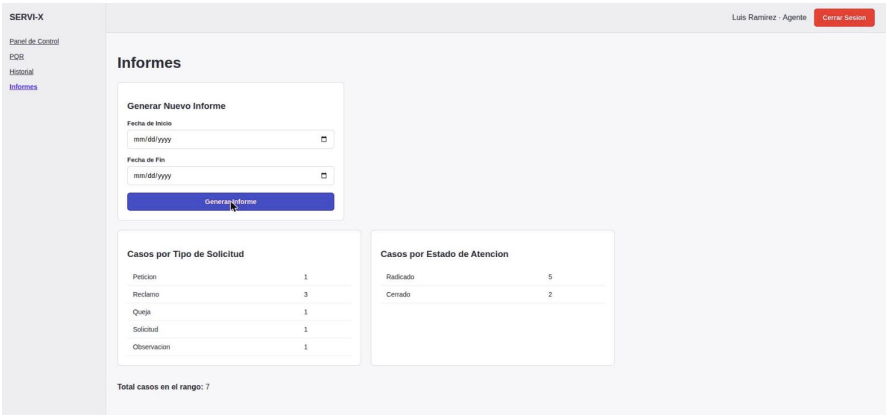
Evidencia (captura de pantalla real):



CP-09 - Generar reportes de gestion

Objetivo / RF - UC	RF-08 / UC-08
Precondicion	Agente autenticado; existen casos de distintos tipos y estados en el sistema.
Datos de entrada	Sin filtro de fecha (rango abierto).
Pasos ejecutados	1) Menu "Informes". 2) Clic en "Generar Informe".
Resultado esperado	El sistema agrupa los casos por tipo de solicitud y por estado de atencion mediante una consulta GROUP BY en PostgreSQL, mostrando el total.
Resultado obtenido	Igual al esperado: se mostraron las tablas "Casos por Tipo de Solicitud" y "Casos por Estado de Atencion" con el total de 7 casos.
Estado	PASS
Error encontrado	Ninguno.
Correccion realizada	No aplica.

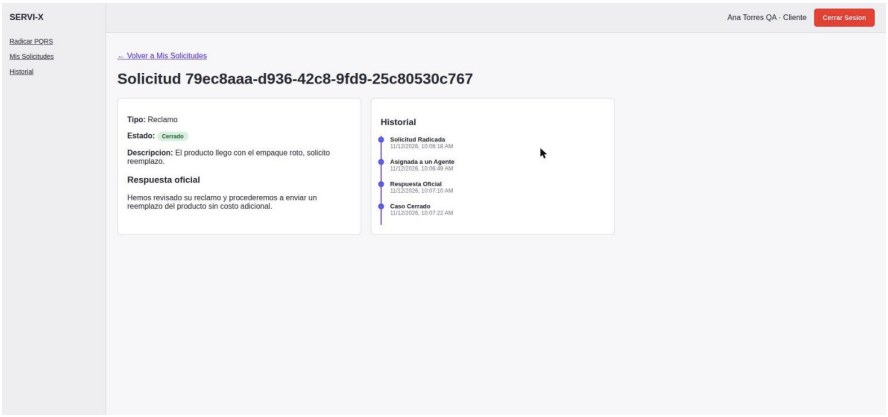
Evidencia (captura de pantalla real):



CP-10 - El Cliente consulta la respuesta oficial de su caso (cierre del ciclo)

Objetivo / RF - UC	RF-03, RF-04 / UC-03, UC-04
Precondicion	Caso cerrado con respuesta oficial (CP-08).
Datos de entrada	Ninguno.
Pasos ejecutados	1) Iniciar sesion como Ana Torres QA (Cliente). 2) Abrir "Mis Solicitudes". 3) Clic en "Ver Detalles" del caso.
Resultado esperado	El Cliente visualiza el estado "Cerrado", el texto completo de la respuesta oficial, y la linea de tiempo completa (Solicitud Radicada, Asignada a un Agente, Respuesta Oficial, Caso Cerrado), confirmando la integracion de extremo a extremo entre frontend, API y base de datos.
Resultado obtenido	Igual al esperado.
Estado	PASS
Error encontrado	Ninguno.
Correccion realizada	No aplica.

Evidencia (captura de pantalla real):



11. Anexo: Script SQL del Esquema de Base de Datos

El siguiente script corresponde al esquema exacto implementado en PostgreSQL 16, obtenido mediante *pg_dump --schema-only* sobre la base de datos real del modulo (sin datos, solo definicion de estructura). Se omite la tabla interna _prisma_migrations, propia del ORM, por no formar parte del modelo de dominio.

```
-- Tipo enumerado para el rol operativo de los responsables (Agente, Administrador)
CREATE TYPE public."Rol" AS ENUM (
'Agente',
'Administrador'
);

-- Funcion que implementa la regla append-only de historial_atencion (RNF-05)
CREATE FUNCTION public.historial_atencion_bloquear_mutacion() RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
RAISE EXCEPTION 'historial_atencion es append-only: no se permiten UPDATE ni DELETE';
END;
$$;

-- Entidad Cliente (usuario final que radica PQRS, UC-01)
CREATE TABLE public.cliente (
cliente_id uuid NOT NULL,
nombre_completo character varying(150) NOT NULL,
tipo_documento character varying(50) NOT NULL,
numero_documento character varying(50) NOT NULL,
email character varying(100) NOT NULL,
password_hash character varying(255) NOT NULL,
fecha_creacion timestamp with time zone DEFAULT CURRENT_TIMESTAMP NOT NULL
);

-- Entidad Responsable (agentes y administradores internos)
CREATE TABLE public.responsable (
responsable_id uuid NOT NULL,
nombre_completo character varying(150) NOT NULL,
correo_corporativo character varying(100) NOT NULL,
password_hash character varying(255) NOT NULL,
rol_operativo public."Rol" NOT NULL,
estado_activo boolean DEFAULT true NOT NULL
);

-- Entidad Caso (nucleo transaccional de la PQRS)
CREATE TABLE public.caso (
caso_id uuid NOT NULL,
cliente_id uuid NOT NULL,
pedido_id character varying(100),
despacho_id character varying(100),
tipo_solicitud character varying(50) NOT NULL,
descripcion text NOT NULL,
estado_atencion character varying(50) DEFAULT 'Radicado'::character varying NOT NULL,
responsable_id uuid,
fecha_creacion timestamp with time zone DEFAULT CURRENT_TIMESTAMP NOT NULL,
fecha_cierre timestamp with time zone
);
```

```
-- Entidad Historial_Atencion (bitacora inmutable de auditoria, RN-04)
CREATE TABLE public.historial_atencion (
historial_id uuid NOT NULL,
caso_id uuid NOT NULL,
estado_anterior character varying(50),
estado_nuevo character varying(50) NOT NULL,
accion_realizada text NOT NULL,
usuario_accion_id uuid NOT NULL,
accion character varying(50) NOT NULL,
fecha_registro timestamp with time zone DEFAULT CURRENT_TIMESTAMP NOT NULL
);

-- Llaves primarias
ALTER TABLE ONLY public.cliente
ADD CONSTRAINT cliente_pkey PRIMARY KEY (cliente_id);
ALTER TABLE ONLY public.responsable
ADD CONSTRAINT responsable_pkey PRIMARY KEY (responsable_id);
ALTER TABLE ONLY public.caso
ADD CONSTRAINT caso_pkey PRIMARY KEY (caso_id);
ALTER TABLE ONLY public.historial_atencion
ADD CONSTRAINT historial_atencion_pkey PRIMARY KEY (historial_id);

-- Indices unicos (RN-01 / unicidad de credenciales)
CREATE UNIQUE INDEX cliente_email_key ON public.cliente USING btree (email);
CREATE UNIQUE INDEX responsable_correo_corporativo_key ON public.responsable USING btree
(correo_corporativo);

-- Trigger append-only (RNF-05): bloquea UPDATE y DELETE sobre historial_atencion
CREATE TRIGGER trg_historial_atencion_append_only
BEFORE DELETE OR UPDATE ON public.historial_atencion
FOR EACH ROW EXECUTE FUNCTION public.historial_atencion_bloquear_mutacion();

-- Llaves foraneas
ALTER TABLE ONLY public.caso
ADD CONSTRAINT caso_cliente_id_fkey FOREIGN KEY (cliente_id)
REFERENCES public.cliente(cliente_id) ON UPDATE CASCADE ON DELETE RESTRICT;
ALTER TABLE ONLY public.caso
ADD CONSTRAINT caso_responsable_id_fkey FOREIGN KEY (responsable_id)
REFERENCES public.responsable(responsable_id) ON UPDATE CASCADE ON DELETE SET NULL;
ALTER TABLE ONLY public.historial_atencion
ADD CONSTRAINT historial_atencion_caso_id_fkey FOREIGN KEY (caso_id)
REFERENCES public.caso(caso_id) ON UPDATE CASCADE ON DELETE RESTRICT;
```